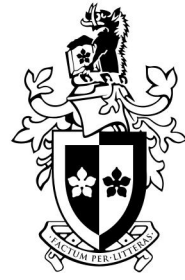


School of Science, Computing and Engineering Technologies
Software Architecture and Application
Assignment 3



Smart Fleet Management (SmartFM) System Technical Report

Lecturer: Mr. Pham Thai Ky Trung

Course: Software Architecture and Application

CourseID: SWE30003

Dates: Mondays, 13:00 - 16:00 p.m

Team Members: Lu Nhat Hoang - 105234956

Duong Nguyen Dang - 1055084444

Truong Ngoc Gia Hieu - 105565520

Start Date: Jul 22th, 2026

End Date: Aug 5th, 2026

Submission Date: Aug 5th, 2026

Word Count: 4710 words

Abstraction

The third document outlines the architecture, design, and implementation of SmartFM, a web-based Fleet Management System engineered to enhance operational transparency and supply chain visibility. Addressing key industry challenges such as real-time vehicle tracking, route monitoring, and multi-role data governance, SmartFM provides a unified platform built upon a modern, decoupled web architecture. Specifically, the system integrates a dynamic React single-page application (SPA) on the front-end with an asynchronous, event-driven Node.js RESTful API services layer on the back-end.

In order to accommodate diverse operational stakeholders, SmartFM features three role-tailored execution portals: an Admin Workspace for dispatchers to manage fleet telemetry and system configurations, a Driver Workspace for field staff to handle route manifests and operational updates, and a Customer Portal providing clients with live order tracking and financial insights. Furthermore, during the prototyping phase, dynamic mock data simulation engines were implemented to validate high-frequency telemetry streams—including GPS coordinates, speed metrics, geofence status, and automated alert logs—prior to physical IoT device integration.

Consequently, system testing and architectural verification confirm that SmartFM successfully achieves efficient state management, strict role-based access control, and low-latency UI updates. Ultimately, the modular design and clean separation of concerns establish a scalable foundation fully prepared for future IoT hardware onboarding, predictive analytics, and cloud-based deployment.

Table of Contents

Part A: Requirement Redesign Analysis	4
1.1 Case Study Requirement Analysis	4
1.2 Refined Overall Class Diagram	4
1.2.1 Admin Subsystem Class Diagram	5
1.2.2 Staff Subsystem Class Diagram	6
1.2.3 Customer Subsystem Class Diagram	7
1.2.4 Design Changes and Rationale	8
Part B: Updated Sequence Diagram	9
2.1 System Bootstrap and Initialization	9
2.2 Customer Order Placement and Validation	10
2.3 Trip Creation and Asset Allocation	11
2.4 Live Telemetry Processing and Automated Alerts	12
Part C: Database Architecture and Schema Design	13
Part D: GitHub Repository Structure	14
Part E: User Interface and User Experience (UI/UX)	16
5.1 Login Screen	16
5.2 Admin Screen	17
5.3 Staff Screen	18
5.4 Customer Screen	19
Part F: Implementation and Technology Stack	20
6.1 Technology Selection and Justifications	20
6.2 Architecture and Data Workflow	21
Part G: Verification and Testing Strategy	21
7.1 Unit and Integration Testing	21
7.2 Role-Based Access Control (RBAC) Testing	22
7.3 Mock Data Telemetry Simulation	22
Part H: Reflection and Conclusion	22

Part A: Requirement Redesign Analysis

1.1 Case Study Requirement Analysis

First and foremost, the third assignment requires students to transition the architectural design from the second assignment into a fully functional software implementation. Consequently, the team members decided to re-evaluate the identified system roles, problem domain, functional requirements, and task descriptions to ensure that our proposed solutions effectively address the challenges ABC-Trans is facing. During the re-analysis phase, team members noticed that three specific roles were mentioned in the case study, including platform administrators (admin), company staff, and customers. By addressing these identified design flaws, the team refined and reconstructed the overall class diagram to ensure comprehensive system coverage and proper role-based architecture, guaranteeing that all roles are included.

1.2 Refined Overall Class Diagram

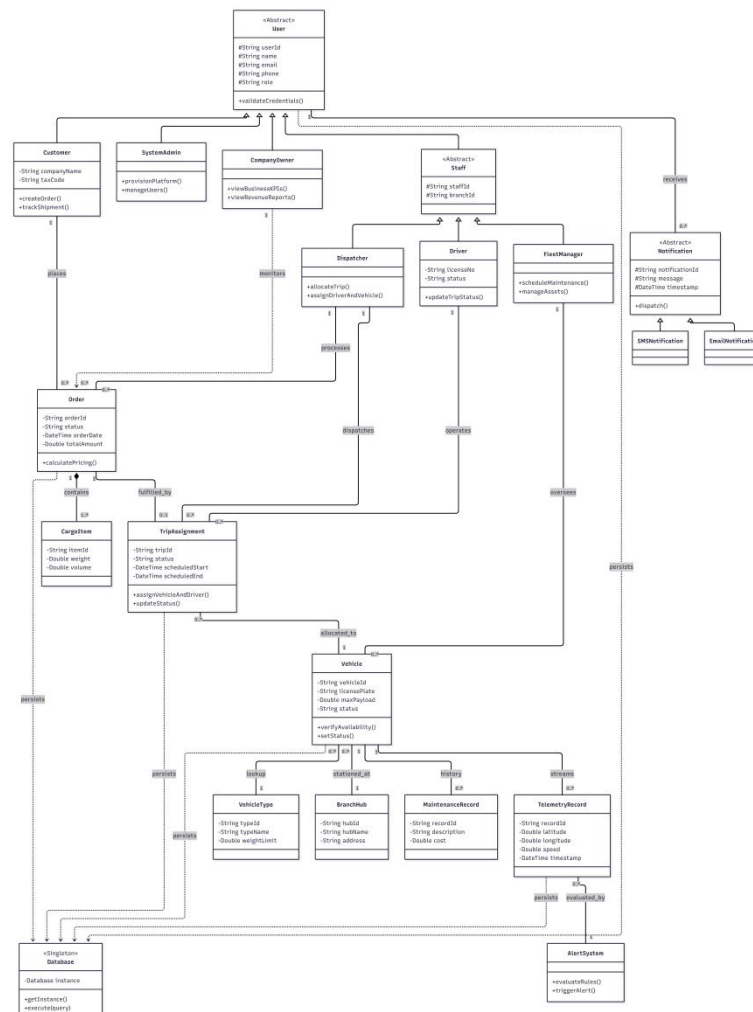


Figure 1: Refined Overall Class Diagram

The Refined Overall Class Diagram above establishes an inheritance hierarchy anchored by the abstract *User* base class, encapsulating core identity attributes (*userId*, *name*, *email*, *phone*, *role*) and defining specialized roles including *Customer*, *SystemAdmin*, *CompanyOwner*, and *Staff*. Furthermore, the *Staff* class is subclassed into operational roles comprising *Dispatcher*, *Driver*, and *FleetManager* which allows fine-grained responsibility assignment. In addition, the information transformation is handled by the abstract *Notification* class, dispatching alerts via its concrete *SMSNotification* and *EmailNotification* implementations directly to relevant user entities.

Next, regarding core components of business domain, a *Customer* places one or many *Order* instances, each containing specific *CargoItem* details and triggering real-time pricing calculations named *calculatePricing()*. Moreover, my team decided to implement a *Dispatcher* for processing orders into *TripAssignment* objects which group orders and match them to specific *Vehicle* entities operated by a *Driver*. Notably, operations are supported by structural entities such as *TransitHub* for route processing and *VehicleType* for capacity constraints while a *FleetManager* oversees vehicle health through *MaintenanceRecord* logs.

In terms of operational monitoring and persistence, active *Vehicle* instances continuously stream spatial and status data into *TelemetryRecord* objects (*latitude*, *longitude*, *speed*, and *timestamp*). Consequently, these records feed directly into an *AlertSystem* that evaluates system rules to trigger automated notifications back to users upon threshold violations or incidents. Finally, important domain entities such as *Order*, *TripAssignment*, *Vehicle*, and *TelemetryRecord* maintain persistence associations with a centralized *Database* class implemented via the Singleton design pattern.

1.2.1 Admin Subsystem Class Diagram

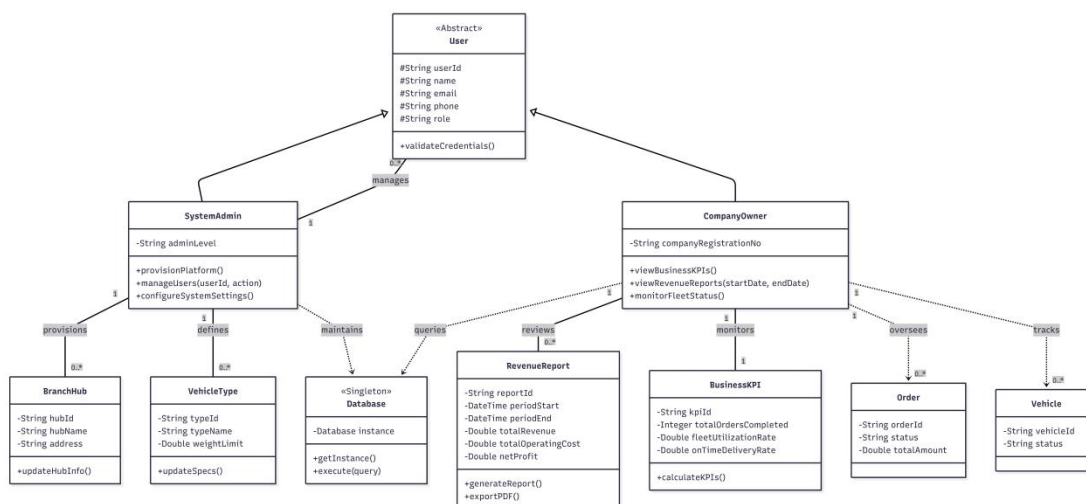


Figure 2: Admin Subsystem Class Diagram

The *Figure 2* illustrates an inheritance hierarchy rooted at the abstract *User* class, showcasing that fundamental identity attributes such as *userId*, *name*, *email*, *phone*, and *role* are encapsulated and *validateCredentials()*

is defined. In detail, the *SystemAdmin* incorporates an *adminLevel* attribute and manages system configuration through operations such as *provisionPlatform()*, *manageUsers()*, and *configureSystemSettings()*. In terms of system maintenance, *SystemAdmin* directly manages lower-level administrative entities by provisioning multiple *BranchHub* instances, defining various *VehicleType* specifications, and maintaining connection with a *Database* singleton to execute administrative queries.

1.2.2 Staff Subsystem Class Diagram

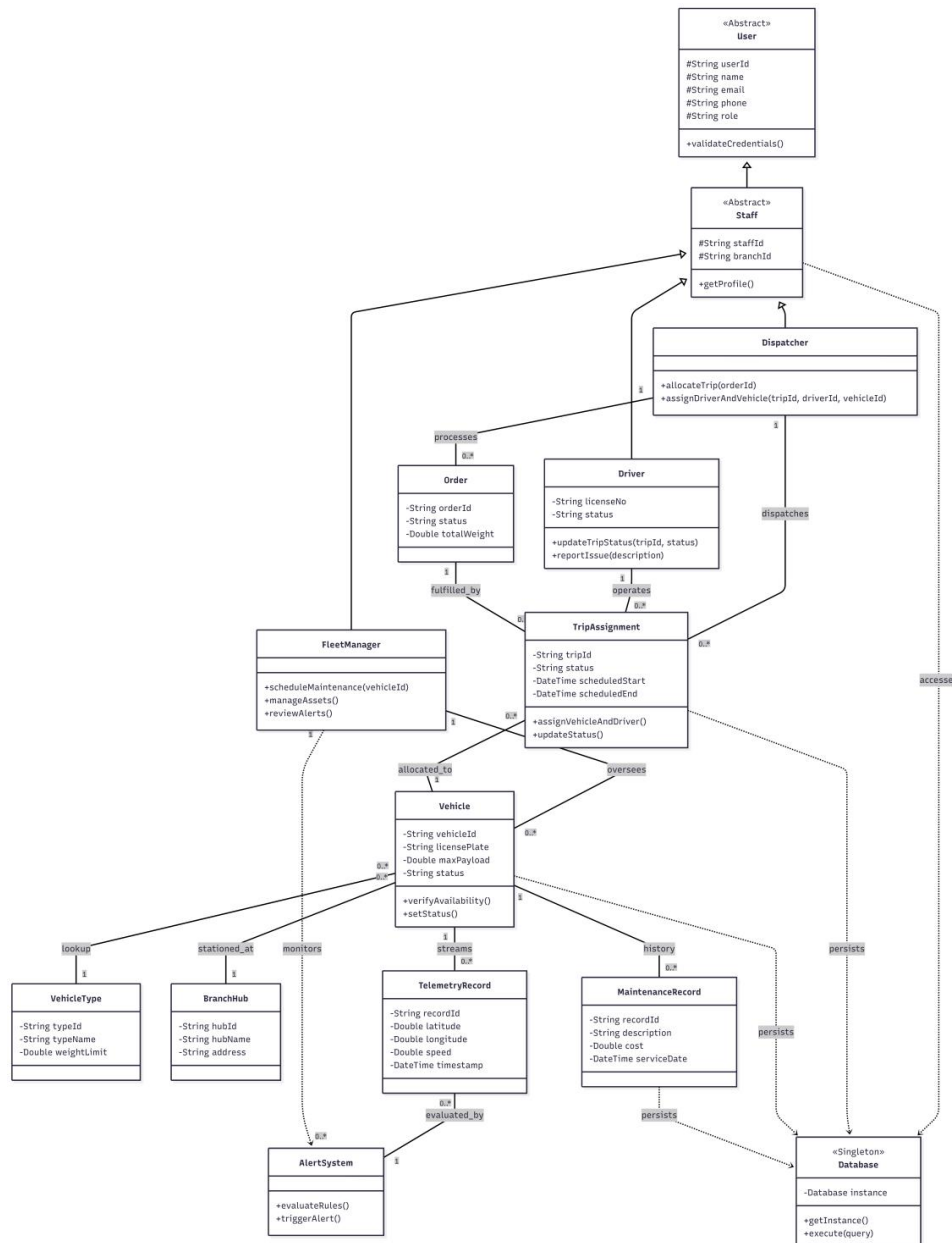


Figure 3: Staff Subsystem Class Diagram

Secondly, **Figure 3** indicates the staff subsystem class diagram based on the entire system named Refined Overall Class Diagram. Furthermore, the diagram establishes an inheritance hierarchy where the abstract *User* class

is extended by an abstract *Staff* class, introducing *staffId*, *branchId*, and the *getProfile()* method. Particularly, the *Staff* class is subclassed into three operational roles: *Dispatcher*, *Driver*, and *FleetManager*. Additionally, the *Dispatcher* processes *Order* instances and coordinates logistics by executing *allocateTrip()* and *assignDriverAndVehicle()* which creates *TripAssignment* records. In contrast, the *Driver* operates the assigned *Vehicle* to fulfill orders, updating trip statuses via *updateTripStatus()* and reporting issues through *reportIssueDescription()*.

Then, *FleetManager* handles asset maintenance (*scheduleMaintenance()*, *manageAssets()*, *reviewAlerts()*) and assigns assets to *TripAssignment* instances for fleet oversight. Furthermore, each *Vehicle* defined by attributes like *maxPayload* and *licensePlate* is stationed at a *BranchHub* and associated with a *VehicleType*. During operations, vehicles continuously stream spatial data into *TelemetryRecord* objects (*latitude*, *longitude*, *speed*), which are evaluated by the *AlertSystem* to trigger real-time alerts.

1.2.3 Customer Subsystem Class Diagram

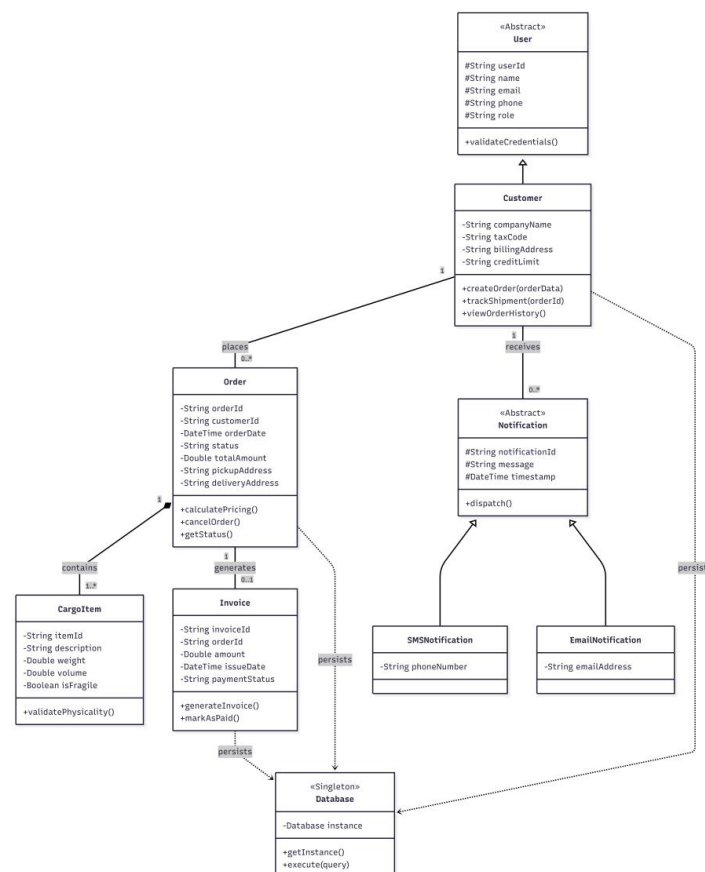


Figure 4: Customer Subsystem Class Diagram

Thirdly, the customer subsystem class diagram inherits from the abstract *User* class, extending it into a concrete *Customer* entity equipped with business-specific attributes such as *companyName*, *taxCode*,

billingAddress, and *creditLimit*. In addition, a *Customer* initiates logistics workflows using operations like *createOrder()*, *trackShipment()*, and *viewOrderHistory()*. When placing an *Order*, the order contains one or more *CargoItem* components (weight, volume, isFragile) validated via *validatePhysicality()* while providing core order management methods such as *calculatePricing()*, *cancelOrder()*, and *getStatus()*.

Following order placement, an *Order* generates an *Invoice* object to track financial metrics and transaction states (amount, paymentStatus) using *generateInvoice()* and *markAsPaid()*. Besides, the *Customer* receives alerts through an abstract *Notification* interface which is specialized into concrete channels: *SMSNotification* (utilizing phoneNumber) and *EmailNotification* (utilizing emailAddress). Finally, core transactional entities including *Customer*, *Order*, and *Invoice* maintain direct persistence relationships with the centralized *Database* singleton.

1.2.4 Design Changes and Rationale

In the Design Changes and Rationale, our team describes what we have changed in the overall class diagram and the reasoning behind these architectural updates:

- Separation of Roles via Specialized Subsystems: Our team suggests splitting into specialized subsystems (*Admin*, *Staff*, and *Customer*) under an abstract *User* base class to avoid a monolithic design and clear up overlapping roles. As a result, the restructured class diagram follows the *Single Responsibility Principle (SRP)* by separating system administration, logistics management, and customer operations into dedicated modules
- Decoupled Telemetry and Automated Alerting: Real-time tracking logic was extracted from the main *Vehicle* class and re-architected into independent *TelemetryRecord* streams processed by an *AlertSystem*. By separating continuous location data (*latitude*, *longitude*, *speed*) from basic asset details, the system avoids memory performance bottlenecks and enables instant rule evaluations for automated alerts.
- Extensible Notification Framework: The communication layer was redesigned around an abstract *Notification* class with concrete implementations for *SMSNotification* and *EmailNotification*. By applying the Open/Closed Principle (OCP), our platform ensures that new messaging channels can be integrated in future iterations without modifying existing customer or order processing code.
- Clear Order, Cargo, and Invoice Lifecycle: Order management was refined by establishing explicit composition relationships with *CargoItem* and an associative connection with *Invoice*. Consequently, our team believes that this structural change separates physical shipment characteristics (such as fragility and volume) from financial status tracking (*markAsPaid()*), providing clearer domain boundaries and accurate state transitions.

- Centralized Data Access via Singleton Pattern: Persistence associations across domain entities were standardized to connect through a single Database instance. Furthermore, the intelligent website utilizing the Singleton Pattern ensures predictable state persistence, prevents redundant connections, and maintains strict execution boundaries for all database queries across the system

Part B: Updated Sequence Diagram

In the second part, our team features specific architectural updates, new entities, and refined relationships introduced in the third assignment, without repeating unchanged legacy components.

2.1 System Bootstrap and Initialization

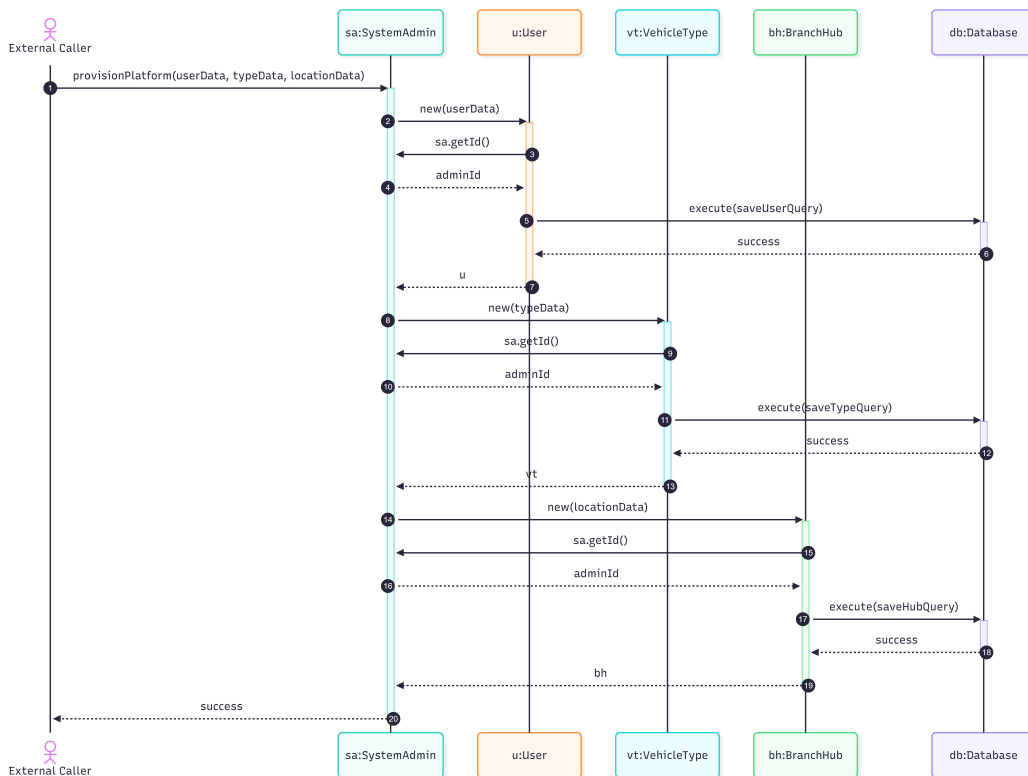


Figure 5: System Bootstrap and Initialization Sequence Diagram

As illustrated in **Figure 5**, the System Bootstrap and Initialization sequence diagram depicts the automated platform setup workflow initiated by a System Administrator to establish core system entities during platform initialization. First, execution starts when an External Caller invokes the `provisionPlatform(userData, typeData, locationData)` method on the **sa:SystemAdmin** controller. After receiving the command, sa:SystemAdmin instantiates a new **u:User** object using the provided user parameters. After that, the instance verifies the administrator's identity via `sa.getId()` before persisting

its record into the centralized **db:Database** singleton through **execute(saveUserQuery)** and returning the created user object **u**.

Next, **sa:SystemAdmin** proceeds to define fleet parameters by instantiating a **vt:VehicleType** object with typeData. Similar to the previous entity, **vt:VehicleType** validates admin authorization, registers vehicle specifications into **db:Database** via **execute(saveTypeQuery)**, and returns the persisted vt instance. Finally, **sa:SystemAdmin** initializes facility infrastructure by creating a **bh:BranchHub** object using locationData. In addition, the **bh:BranchHub** verifies administrative credentials, persists hub location details into **db:Database** via **execute(saveHubQuery)**, and returns the bh reference. Once all primary domain entities (users, vehicle specifications, and operational hubs) are successfully instantiated and saved, **sa:SystemAdmin** emits a final success response back to the External Caller, completing the initialization sequence.

2.2 Customer Order Placement and Validation

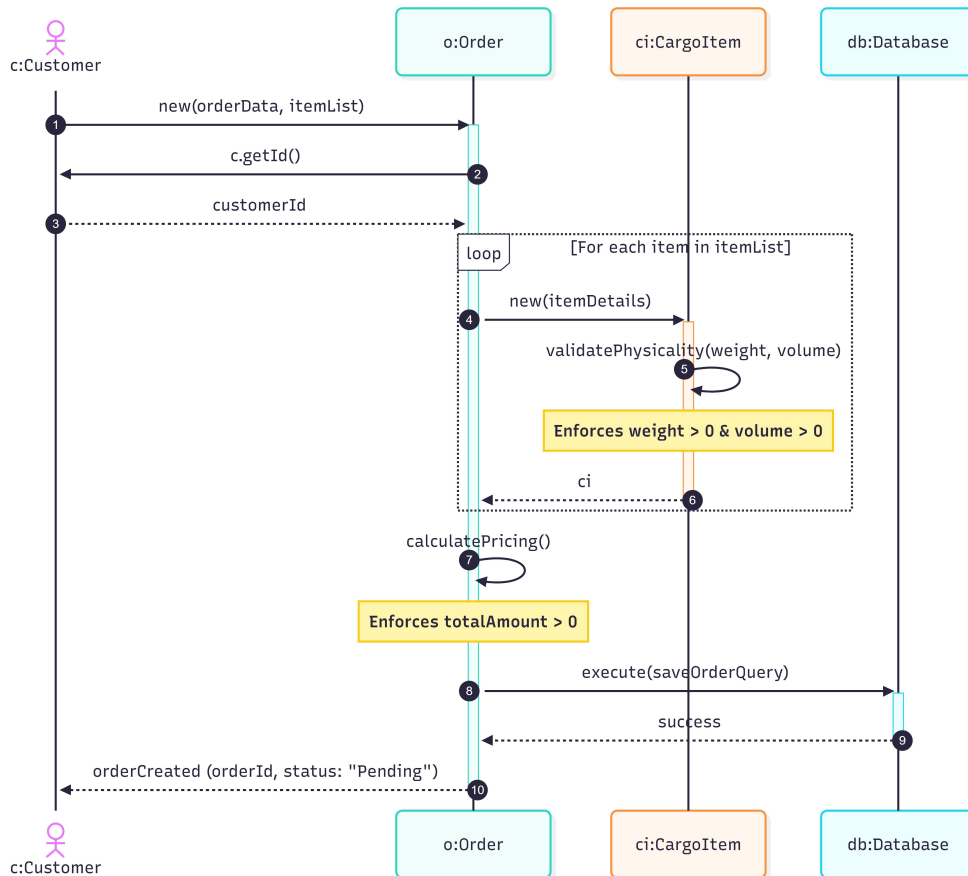


Figure 6: Customer Order Placement and Validation Sequence Diagram

It is clear that **Figure 6** highlights the Customer Order Placement and Validation sequence diagram depicts the transactional workflow for creating and verifying a new shipment order. Firstly, execution starts when a **c:Customer** invokes the **new(orderData, itemList)** constructor on the **o:Order**

entity. Upon receiving the initialization request, *o:Order* retrieves the customer's identifier via *c.getId()* to associate ownership. After that, *o:Order* enters an iterative loop for each item in *itemList*, instantiating a corresponding *ci:CargoItem* object with *new(itemDetails)*. During this iteration, each *ci:CargoItem* executes *validatePhysicality(weight, volume)* to enforce domain business rules, specifically ensuring that physical constraints are met ($\text{weight} > 0$ and $\text{volume} > 0$).

Then, once all cargo items are validated and returned, *o:Order* invokes its internal *calculatePricing()* method to compute the total order amount, enforcing that the calculated *totalAmount* is greater than zero. Similar to the previous system processes, *o:Order* then interacts with the database layer by executing *execute(saveOrderQuery)* on the centralized *db:Database* singleton. Finally, upon receiving a success confirmation from *db:Database*, *o:Order* returns an *orderCreated* response containing the generated *orderId* and an initial status of "Pending" back to the *c:Customer*, completing the order creation lifecycle.

2.3 Trip Creation and Asset Allocation

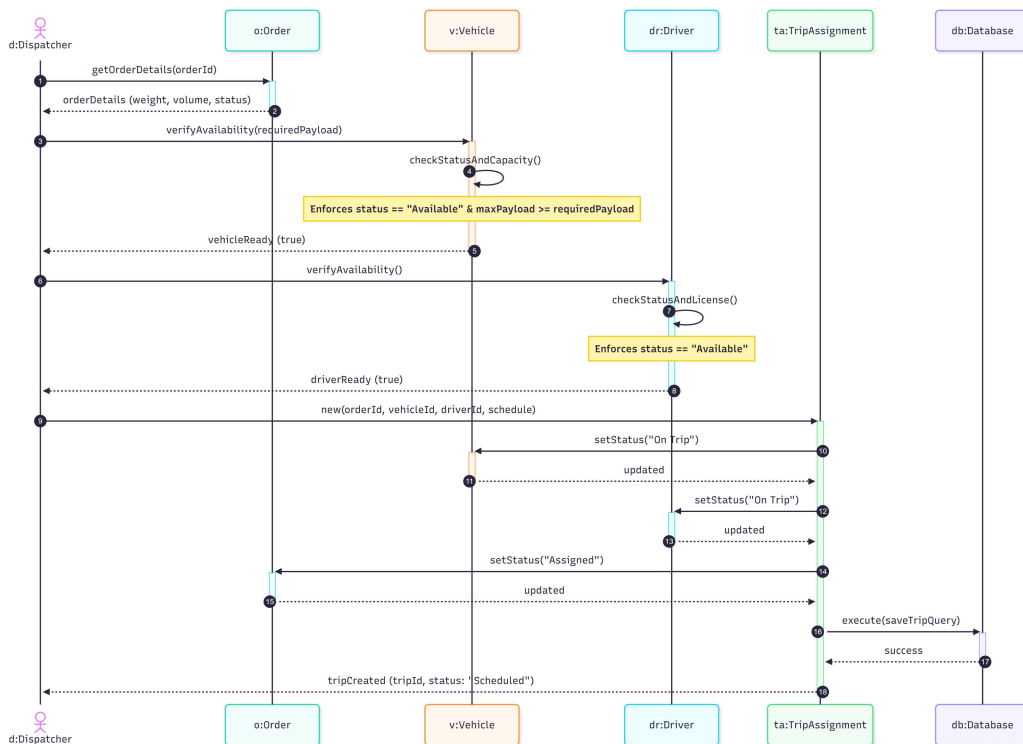


Figure 7: Trip Creation and Asset Allocation Sequence Diagram

As illustrated in *Figure 7*, sequence diagram of Trip Creation and Asset Allocation depicts the operational workflow performed by a Dispatcher to assign suitable logistics assets to a pending order. First, execution starts when a *d:Dispatcher* queries order requirements by invoking

getOrderDetails(orderId) on the target **o:Order** entity, which returns the shipment's weight, volume, and status. Upon retrieving these specifications, **d:Dispatcher** verifies asset availability by invoking verifyAvailability(requiredPayload) on a candidate **v:Vehicle**. Consequently, **v:Vehicle** executes an internal check checkStatusAndCapacity() to enforce business constraints, ensuring its status is "Available" and its maximum payload capacity satisfies the required payload, before returning a vehicleReady(true) signal.

Next, **d:Dispatcher** validates personnel readiness by invoking verifyAvailability() on a candidate **dr:Driver**. Subsequently, **dr:Driver** runs checkStatusAndLicense() to ensure their operational status is "Available" and credentials are valid, returning a driverReady(true) confirmation. Following both asset and driver availability confirmations, **d:Dispatcher** instantiates a new **ta:TripAssignment** object with the assigned order, vehicle, driver, and schedule details. Immediately after, **ta:TripAssignment** updates the operational status of **v:Vehicle** and **dr:Driver** to "On Trip", while updating **o:Order** to "Assigned". Finally, **ta:TripAssignment** persists the assignment record into the centralized **db:Database** singleton via execute(saveTripQuery) and returns a tripCreated response with a "Scheduled" status back to **d:Dispatcher**, completing the resource allocation process.

2.4 Live Telemetry Processing and Automated Alerts

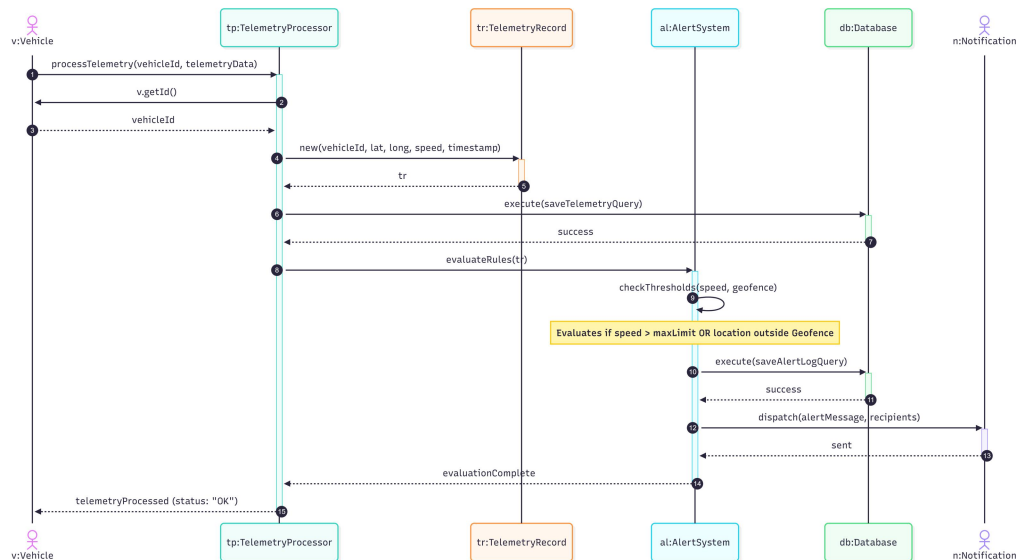


Figure 8: Live Telemetry Processing and Automated Alerts

As illustrated in **Figure 8**, sequence diagram of Live Telemetry Processing and Automated Alerts depicts the real-time monitoring and event-driven alerting workflow for operational fleet tracking. First, execution starts when a **v:Vehicle** transmits continuous positional streaming data by calling processTelemetry(vehicleId, telemetryData) on the **tp:TelemetryProcessor**

component. Upon receiving this call, **tp:TelemetryProcessor** retrieves the vehicle identifier via **v.getId()** and instantiates a new **tr:TelemetryRecord** object containing spatial attributes (lat, long, speed, and timestamp). Immediately after, **tp:TelemetryProcessor** persists this spatial log directly into the centralized **db:Database** singleton through **execute(saveTelemetryQuery)**.

Next, **tp:TelemetryProcessor** forwards the telemetry record to **at:AlertSystem** by calling **evaluateRules(tr)** to analyze operational parameters. Consequently, **at:AlertSystem** performs an internal check **checkThresholds(speed, geofence)** to evaluate if current vehicle metrics exceed speed limits or venture outside assigned geofence boundaries. In the event of a violation, **at:AlertSystem** logs the event by executing **execute(saveAlertLogQuery)** on **db:Database** and dispatches an alert message to the target **n:Notification** service. Finally, upon completing rule evaluations, **at:AlertSystem** returns an **evaluationComplete** signal back to **tp:TelemetryProcessor**, which then confirms the operation to **v:Vehicle** with a **telemetryProcessed** status of "OK", concluding the monitoring loop.

Part C: Database Architecture and Schema Design

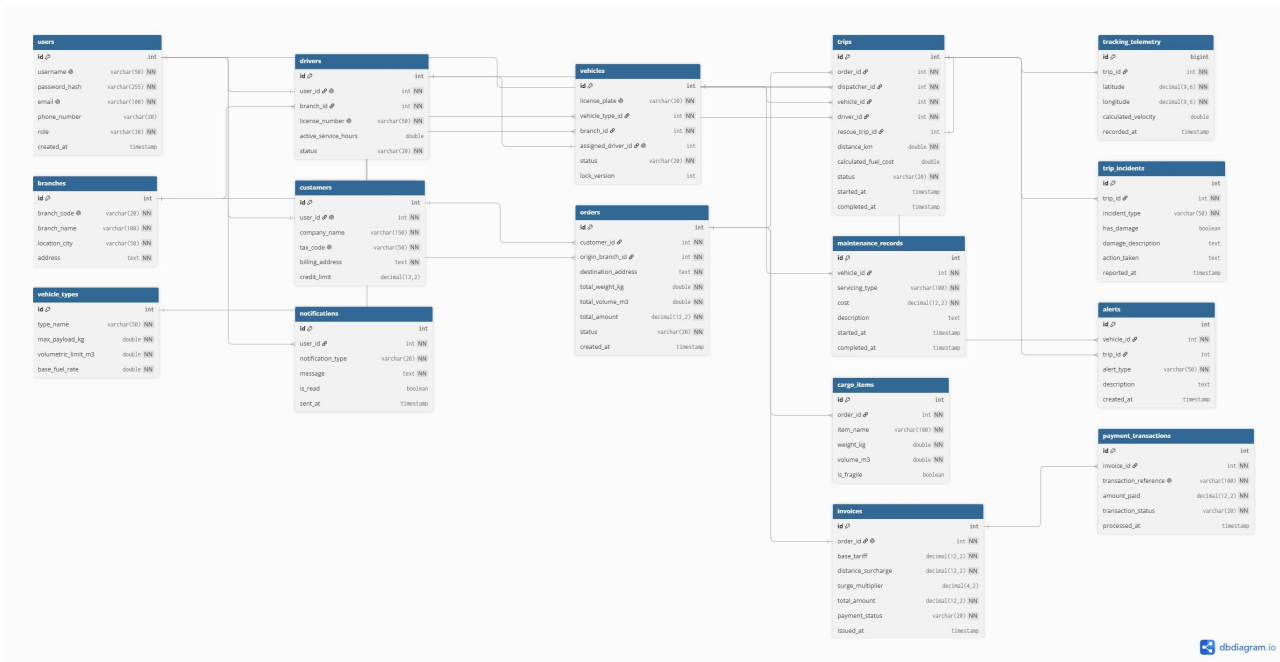


Figure 9: Relational Database Schema for SmartFM System

As illustrated in **Figure 9**, the relational database schema above represents the underlying persistence layer designed to support end-to-end logistics operations, asset management, and live tracking. First and foremost, core user identity and access management are established within the **users** table, which serves as a foundational entity. Subsequently, this core identity is extended to specialized profile entities through one-to-one relationships with both **drivers** and **customers**. In addition, system communications are captured through the **notifications** table, thereby ensuring that multi-channel messages are logged continuously against

specific user IDs. Furthermore, operational infrastructure is organized via **branches** and **vehicle_types** tables, which consequently provide essential structural references for physical hubs, maximum payload capacities, and volumetric restrictions.

In terms of commercial workflows, order processing and financial management are structured around key domain entities. To begin with, primary transaction records are maintained within the **orders** table, which forms explicit relationships with **customers** and **branches**. At the same time, it hosts itemized physical details through the **cargo_items** table. Meanwhile, billing processes are governed by connecting **orders** to **invoices**, which accurately compute pricing breakdowns such as base tariffs, distance surcharges, and payment statuses. Consequently, each invoice record connects to **payment_transactions** in order to preserve complete financial audit trails for executed payments.

Regarding dynamic fleet execution, asset allocation and real-time monitoring constitute the operational heart of the schema. To start with, fleet entities stored in **vehicles** are mapped directly to designated **vehicle_types** and assigned drivers, whereas maintenance histories are systematically recorded in **maintenance_records**. Subsequently, trip fulfillment is orchestrated through the **trips** table, thereby binding specific orders, dispatchers, vehicles, and drivers into a unified operational record. For the purpose of real-time tracking, **trips** continuously streams positional logs into **tracking_telemetry** to capture latitude, longitude, and calculated velocity. As a result, operational anomalies are seamlessly categorized into **trip_incidents** for physical damage reports and **alerts** for automated system threshold violations, ultimately forming a cohesive data ecosystem.

Part D: GitHub Repository Structure

Assignments	Finish part Updated Class Diagram	11 hours ago
Handbook	Add Week 2 Note	3 months ago
Modules	Add Week Presentation	last week
Cheatsheet.docx	Add extra activities and slide presentation	3 months ago
README.md	Revise README for better readability and organization	3 days ago
Syllabus.pdf	Preparation	3 months ago

Figure 10: Github Repository Structure

As shown in *Figure 10*, our team generates a repository named **Software_Architecture_and_Design_SWE30003** including following sub-folders and files:

- Assignments/ sub-folder: used for storing important project deliverables, design diagrams, and final report submissions.

- Handbook/ sub-folder: used for containing recommended books for reading while studying the course.
- Modules sub-folder: used for saving weekly presentation slides, seminar topics, and lecture materials
- README.md: provides a comprehensive project overview, layout structure, and repository guidelines for team members.
- Syllabus.pdf file: Outlines essential course requirements, learning outcomes, and administrative guidelines, thereby finalizing a clean and structured project environment.

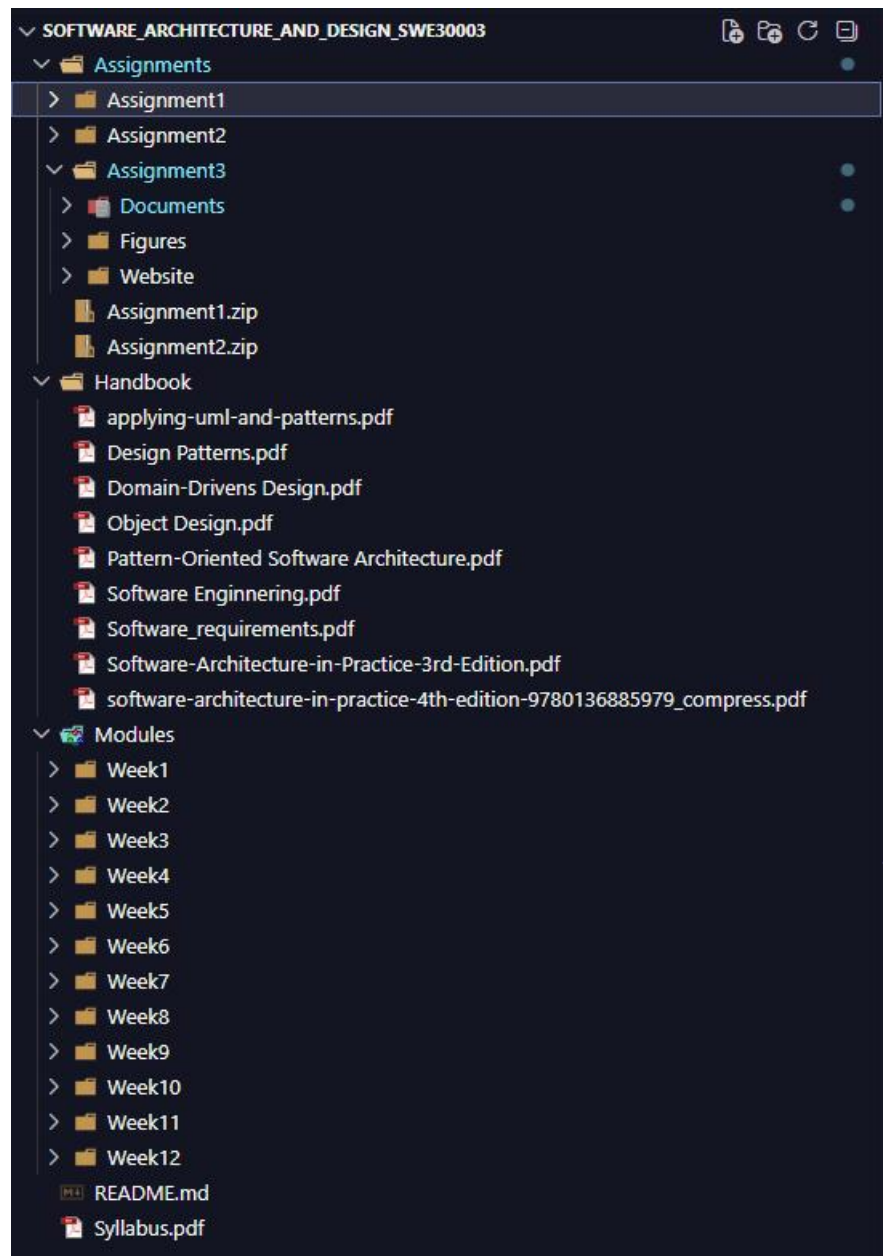


Figure 11: Expanded Directory Tree of the SmartFM Repository

In order to provide a comprehensive structural breakdown, Figure 11 displays the fully expanded tree structure for key sub-folders within the repository:

- **Assignments/** directory: First and foremost, project work is organized sequentially into Assignment1, Assignment2, and Assignment3 folders. In addition, archived source deliverables such as Assignment1.zip and Assignment2.zip are retained for historical submission verification. Inside Assignment3, artifacts are systematically segregated into Documents for reports, Figures for architectural and sequence diagrams, and Website for relevant web source code.
- **Handbook/** directory: Subsequently, foundational reference literature is centralized within this folder. Specifically, core textbooks including *Applying UML and Patterns*, *Design Patterns*, *Domain-Driven Design*, *Object Design*, *Pattern-Oriented Software Architecture*, and various editions of *Software Architecture in Practice* are stored directly in PDF format for easy academic reference.
- **Modules/** directory: Furthermore, weekly learning materials are chronologically categorized from Week1 through Week12. Consequently, lecture slides, lab tutorials, and weekly exercises are isolated by academic week to support structured study progression.
- **Root Configuration Files:** Finally, essential administrative resources such as README.md and Syllabus.pdf remain positioned at the root level, thereby providing immediate access to repository setup rules and course guideline

Part E: User Interface and User Experience (UI/UX)

5.1 Login Screen

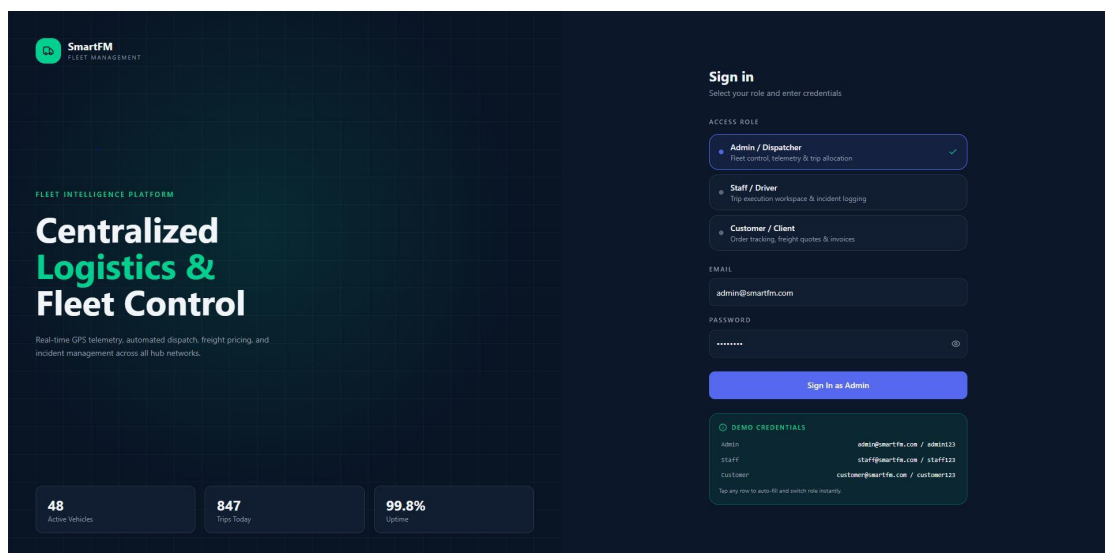


Figure 12: SmartFM Centralized Login Screen

Serving as the primary entrance to the SmartFM platform, *Figure 12* showcases an intuitive authentication portal built on a modern React front-end framework integrated with a responsive Node.js backend architecture. In order to facilitate

an efficient, user-centric onboarding experience, several key components have been implemented:

- **Role-Based Access Selection:** First and foremost, users can directly select their target operational role (such as Admin or Dispatcher, Staff or Driver, or Customer or Client) before inputting credentials, thereby enabling immediate dynamic routing to role-specific dashboards.
- **Mock Data Metrics and Live Indicators:** Subsequently, real-time performance indicators positioned in the bottom-left panel (highlighting active vehicles, daily trip counts, and system uptime) are populated using realistic mock data, which effectively simulates live operational telemetry during prototype evaluation.
- **Pre-configured Demo Credentials:** In addition, an interactive Demo Credentials card is seamlessly embedded on the right-hand side, allowing testing teams to instantly auto-fill credentials and evaluate cross-role functionalities without tedious manual entry.
- **Modern Dark-Mode Aesthetic:** Meanwhile, high-contrast dark visual styling, clear field highlights, and explicit state responses come together to minimize visual fatigue for fleet managers operating under high-density workflows.

5.2 Admin Screen

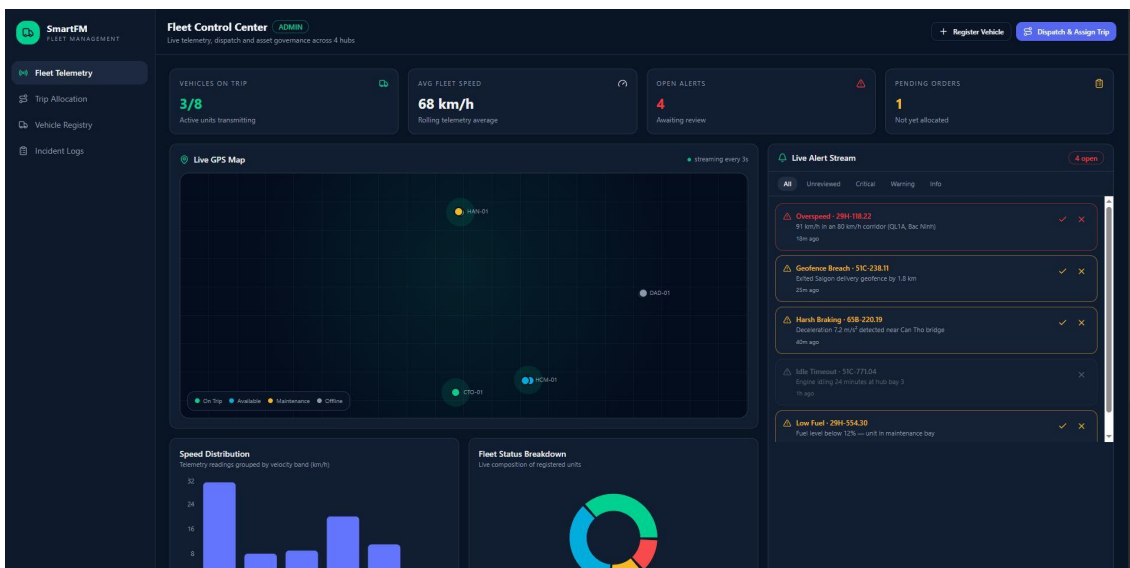


Figure 13: SmartFM Fleet Control Center (Admin Dashboard)

Designed to give fleet operators complete oversight of ongoing operations, **Figure 13** showcases the main control portal brought to life using a modern React interface and driven by a scalable Node.js backend architecture. In order to streamline complex logistics management, several key interactive features have been integrated into this view:

- **Top-Level Fleet Telemetry Summary:** Right at the top, a series of quick-reference cards highlights core metrics such as active vehicle counts, average fleet velocity, pending alerts, and unassigned orders, which are populated via simulated mock data to reflect real-time operational status.
- **Interactive Live GPS Map:** Positioned in the center, a live spatial grid maps out current vehicle positions across key hubs (including HAN-01, DAD-01, HCM-01, and CTO-01) alongside clear color codes that indicate whether a unit is on a trip, available, offline, or under maintenance.
- **Live Alert Stream and Event Logging:** Situated on the right-hand side, a dedicated event feed captures safety and maintenance triggers (ranging from overspeeding incidents and geofence breaches to harsh braking and low fuel warnings), enabling dispatchers to acknowledge or resolve issues on the fly.
- **Analytics and Fleet Composition Breakdowns:** Anchoring the bottom section, dual graphical modules break down telemetry data into actionable insights, featuring speed distribution bar charts alongside a donut chart showing overall fleet availability.

5.3 Staff Screen

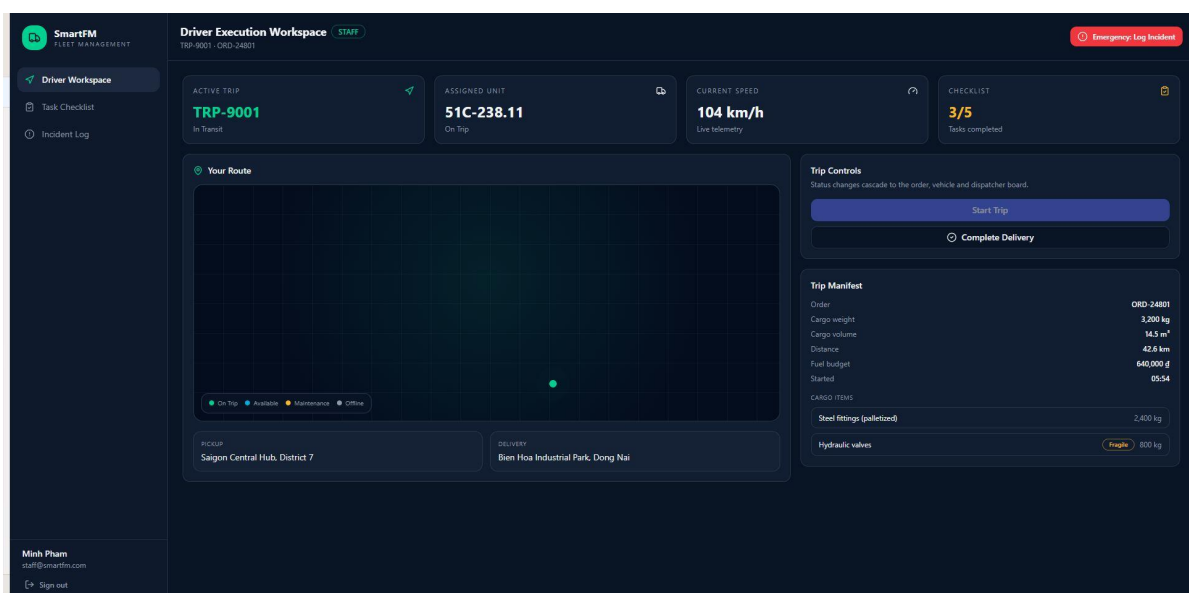


Figure 14: SmartFM Driver Execution Workspace

Tailored specifically for field staff and vehicle operators, Figure 14 presents a specialized operational interface constructed using React for seamless state rendering and backed by Node.js services. In order to guide drivers through assigned delivery lifecycles while maintaining real-time telemetry synchronization, several core operational controls have been structured:

- **Active Trip Header and Telemetry Gauges:** Positioned prominently along the top bar, real-time indicators highlight the active trip ID (TRP-9001), assigned vehicle unit (51C-238.11),

- live velocity telemetry (104 km/h), and task completion metrics (3/5 completed), all rendered dynamically via simulated mock data.
- Navigation Map and Route Hubs: Occupying the primary left panel, an interactive spatial interface displays active vehicle routing alongside clear pickup and delivery points (such as Saigon Central Hub, District 7 to Bien Hoa Industrial Park, Dong Nai), giving drivers instant route clarity.
- Trip Controls and Emergency Incident Logging: Located in the upper right section, primary execution buttons allow drivers to update trip status (such as starting trips or completing deliveries) which instantly cascade changes to the dispatcher board, alongside a high-visibility emergency button for immediate incident reporting.
- Detailed Trip Manifest and Cargo Breakdown: Anchoring the lower right panel, comprehensive cargo parameters outline order details (including order ID, weight, volume, route distance, and fuel budget) alongside an itemized cargo list specifying specialized handling requirements such as palletized steel fittings and fragile hydraulic valves.

5.4 Customer Screen

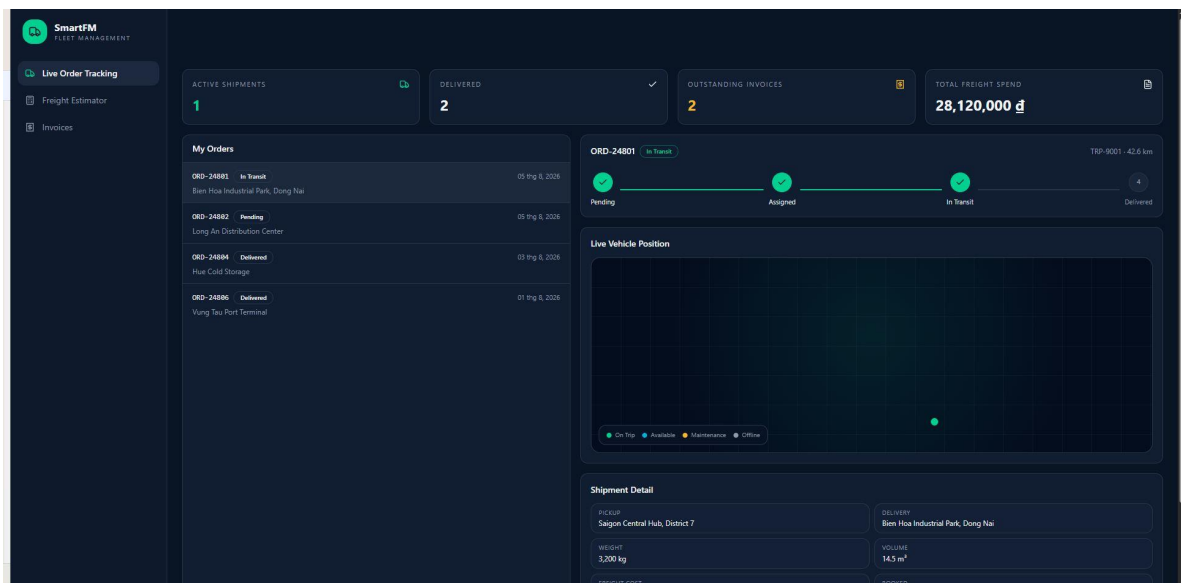


Figure 15: SmartFM Customer Portal Screen

Engineered to deliver complete supply chain transparency for client organizations, Figure 15 showcases the live order tracking dashboard built using a dynamic React front-end coupled with a real-time Node.js API backend. In order to keep clients informed throughout the delivery lifecycle without needing direct dispatcher intervention, several intuitive tracking modules have been integrated:

- Top Summary KPI Cards: Highlighted across the top header, client-specific performance metrics (including active shipments, completed

deliveries, pending invoices, and total freight expenditure) are populated dynamically using simulated mock data to provide immediate financial and operational visibility.

- **Order History Navigation:** Positioned on the left side, an organized order list displays active and historical shipments (such as ORD-24801 destined for Bien Hoa Industrial Park and ORD-24802 for Long An Distribution Center) with real-time status badges like *In Transit*, *Pending*, and *Delivered*.
- **Progress Tracker and Live Vehicle Positioning:** Dominating the upper-right view, a visual milestone stepper tracks real-time progress from *Pending* through *In Transit* to *Delivered*, sitting directly above an interactive GPS map that renders the exact spatial coordinates of the assigned transport unit.
- **Granular Shipment Details:** Located at the bottom right, a structured summary card breaks down essential logistics parameters, explicitly outlining pickup hubs, destination points, cargo weight, volume, freight costs, and booking timestamps.

Part F: Implementation and Technology Stack

In order to transition architectural specifications into a functional software system, the SmartFM platform relies on a modern, decoupled web architecture. By combining a client-side single-page application framework with a scalable server-side execution environment, the system guarantees high responsiveness, modularity, and rapid telemetry data processing across all user roles.

6.1 Technology Selection and Justification

- **Front-End Framework (React):** First and foremost, React was selected to construct the user interface owing to its component-based architecture and efficient Virtual DOM rendering. Consequently, high-frequency dashboard updates (such as live speed tracking, active vehicle GPS coordinates, and real-time alert streams) are re-rendered selectively without triggering full-page refreshes, thereby ensuring an optimal user experience for dispatchers and field operators.
- **Back-End Runtime Environment (Node.js):** Subsequently, Node.js serves as the core backend platform due to its event-driven, non-blocking I/O model. Because fleet telemetry involves continuous streams of status updates and concurrent user requests, the asynchronous event loop of Node.js effectively handles high throughput with low resource overhead.
- **Data Layer and Simulation Strategy:** Furthermore, during the prototyping and architectural verification phase, mock data generators are integrated directly into the Node.js API services. This setup simulates real-time vehicle telemetry (such as velocity distributions, geofence breaches, and route progression) before connecting to physical IoT hardware.

6.2 Architecture and Data Workflow

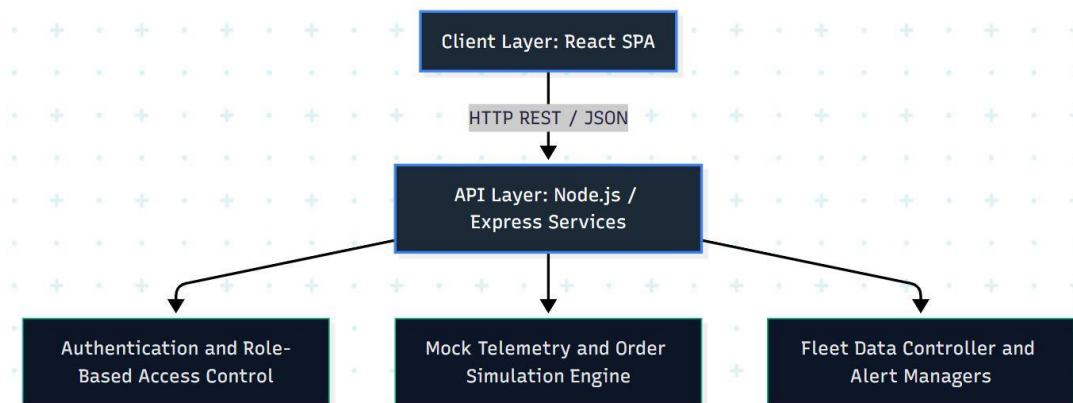


Figure 16: Architecture and Data Workflow

- **Decoupled API-Driven Communication:** As illustrated in the high-level implementation model above, the React client communicates with the Node.js backend exclusively via stateless RESTful API endpoints.
- **Role-Based Routing and State Management:** In addition, incoming requests are authenticated and processed based on the active user role (Admin, Driver, or Customer), which determines the structure of the JSON payload returned to the front-end components.
- **Component-Level Data Binding:** Finally, front-end state hooks seamlessly bind incoming API data to visual elements (including interactive map grids, status indicators, and analytical charts), guaranteeing clean separation between presentation logic and backend business rules.

Part G: Verification and Testing Strategy

In order to guarantee system reliability, architectural integrity, and seamless operational flows across all user roles, a structured verification strategy was designed for the SmartFM platform. The testing process focuses on evaluating API endpoint responses, role-based access security, and component state updates driven by real-time simulated telemetry.

7.1 Unit and Integration Testing

- **Backend API Endpoint Verification:** First and foremost, integration tests validate that the Node.js REST endpoints accurately process incoming requests, handle authentication tokens correctly, and return structured JSON responses with appropriate status codes.
- **Component State Rendering:** Subsequently, unit tests built for the React client confirm that individual UI modules (including spatial tracking maps, KPI summary cards, and live alert streams)

correctly handle state updates without triggering memory leaks or unexpected visual artifacts.

- **Asynchronous Data Handling:** In addition, test suites simulate network delays and payload variations to ensure the asynchronous event loop maintains smooth data flow during high-frequency dashboard updates.

7.2 Role-Based Access Control (RBAC) Testing

- **Admin Clearance Levels:** First and foremost, automated authorization checks confirm that only users authenticated with Admin privileges can access global dispatch controls, system-wide fleet telemetry, and vehicle registry management.
- **Staff and Driver Scoping:** Subsequently, security tests verify that field operators logged into the Driver Execution Workspace are restricted strictly to their assigned trip manifests, route navigation controls, and emergency logging features.
- **Client Data Isolation:** Furthermore, testing ensures that customer organization accounts are limited to viewing their specific active shipments, freight expenditure summaries, and order tracking milestones, thereby preventing cross-tenant data leakage.

7.3 Mock-Data Telemetry Simulation

- **Dynamic Metric Validation:** First and foremost, testing scripts verify that mock data generators within the Node.js layer reliably stream realistic vehicle parameters (such as rolling speed averages, geofence boundary exits, and fuel level drops) at configured intervals.
- **Alert Trigger Verification:** Subsequently, simulated threshold violations (such as overspeeding beyond 80 km/h or harsh deceleration events) are tested to ensure they instantly propagate to the Live Alert Stream component for dispatcher review.
- **Prototype Stability Assessment:** Finally, prolonged execution runs demonstrate that the mock telemetry pipeline maintains steady memory consumption and UI responsiveness, successfully validating the system architecture prior to physical IoT hardware integration.

Part H: Reflection and Conclusion

After successfully engineering the SmartFM platform, key insights were gained regarding the implementation of role-tailored software architectures for real-time telematics. Balancing high-frequency telemetry updates with seamless frontend rendering highlighted the importance of modular component design in React to prevent performance bottlenecks. Furthermore, utilizing mock data to simulate

complex operational workflows (such as live GPS updates, alert streams, and shipment tracking) underscored the necessity of strict API contracts, ensuring the system can transition smoothly to real-world IoT hardware integration without architectural friction.

In conclusion, the SmartFM project successfully fulfills its primary design and architectural objectives, delivering a robust, decoupled system powered by a React frontend and a Node.js backend. By providing distinct, feature-rich workspaces for dispatchers, field staff, and enterprise clients, the system establishes a strong foundation for operational efficiency and supply chain transparency. Ultimately, a scalable infrastructure combined with a clean separation of concerns ensures that the platform is fully prepared for future enhancements, including physical device integration and cloud-based deployment.

References

SWE30003 Software Architecture and Application. (2026). Project assignment 1 – System Analysis and Initial Architecture. Swinburne University of Technology.

SWE30003 Software Architecture and Application. (2026). Project assignment 2 – System Implementation and Verification. Swinburne University of Technology.